



**Instituto Politécnico Nacional**  
Unidad Profesional Interdisciplinaria en  
Ingeniería y Tecnologías Avanzadas

# **Diseño de propuesta de fragmentación para Ejercicio 3**

**Alumnos:** González Vázquez Rodrigo

Mendoza Xicohtencatl Héctor Jair

**Profesor:** De la Cruz Sosa Carlos

**Materia:** Base de datos distribuidas

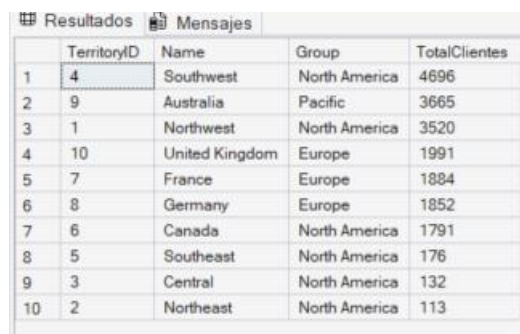
**Grupo:** 3TM2

**Fecha:** 24 de septiembre de 2026

Para iniciar nuestro diseño de fragmentación, en lugar de utilizar un atributo secuencial como **CustomerID**, decidimos trabajar con **TerritoryID** porque agrupar a los clientes por su ubicación geográfica tiene más sentido para la lógica del negocio que agruparlos por su número de registro.

Antes de definir los fragmentos, necesitábamos saber cómo estaban distribuidos los datos. Realizamos una consulta exploratoria sobre el estado actual de la base de datos (que cuenta con más de 19,800 clientes).

```
SELECT
Sales.SalesTerritory.TerritoryID,
Sales.SalesTerritory.Name,
Sales.SalesTerritory.[Group],
COUNT(Sales.Customer.CustomerID) AS TotalClientes
FROM Sales.SalesTerritory
JOIN Sales.Customer
ON Sales.SalesTerritory.TerritoryID = Sales.Customer.TerritoryID
GROUP BY
Sales.SalesTerritory.TerritoryID,
Sales.SalesTerritory.Name,
Sales.SalesTerritory.[Group]
ORDER BY
COUNT(Sales.Customer.CustomerID) DESC;
```



|    | TerritoryID | Name           | Group         | TotalClientes |
|----|-------------|----------------|---------------|---------------|
| 1  | 4           | Southwest      | North America | 4696          |
| 2  | 9           | Australia      | Pacific       | 3665          |
| 3  | 1           | Northwest      | North America | 3520          |
| 4  | 10          | United Kingdom | Europe        | 1991          |
| 5  | 7           | France         | Europe        | 1884          |
| 6  | 8           | Germany        | Europe        | 1852          |
| 7  | 6           | Canada         | North America | 1791          |
| 8  | 5           | Southeast      | North America | 176           |
| 9  | 3           | Central        | North America | 132           |
| 10 | 2           | Northeast      | North America | 113           |

Si fragmentábamos directamente por el nombre de la región, por ejemplo, agrupar todos los territorios bajo 'North America', 'Europe' y 'Pacific'), el resultado era un desbalance: Norteamérica se quedaba con más de registros que el Pacífico 3,600.

Este desbalance rompería uno de nuestros objetivos principales: asegurar una distribución uniforme para no sobrecargar un nodo. Por ello, aplicamos una estrategia de agrupación manual, sumando los totales de cada territorio para crear 5 nodos, con un aproximadamente de 3,900 clientes cada uno.

- **Fragmento 1 (Suroeste):** Territorio 4 = **4,696** clientes.
- **Fragmento 2 (Pacífico y Centro-Noreste):** Territorios 9, 3, 2  
 $(3665 + 132 + 113) = \mathbf{3,910}$  *clientes*.
- **Fragmento 3 (Europa Occidental):** Territorios 10, 7  
 $(1991 + 1884) = \mathbf{3,875}$  *clientes*.
- **Fragmento 4 (Noroeste y Sureste):** Territorios 1, 5  
 $(3520 + 176) = \mathbf{3,696}$  *clientes*.
- **Fragmento 5 (Alemania y Canadá):** Territorios 8, 6  
 $(1852 + 1791) = \mathbf{3,643}$  *clientes*

Una vez definidos los 5 grupos balanceados, procedimos a aplicar el algoritmo **COM\_MIN** para generar nuestros fragmentos primarios. El algoritmo evalúa nuestros **predicados simples** (por ejemplo: TerritoryID = 4 o TerritoryID IN (1, 5)) y genera los llamados **predicados minitérmino**. Estos minitérminos son las condiciones exactas que **restringen** lógicamente qué **filas** de **Sales.Customer** van a cada uno de nuestros 5 nodos, garantizando que los fragmentos sean útiles y uniformes.

Por ejemplo, si queríamos agrupar el ID de territorio 10 con el 7 para formar un nodo, tenemos que definir el predicado minitérmino que contuviera esa combinación, y ahí es donde restringimos la información que usaremos:

```
-- NODO 1: Suroeste
SELECT * FROM Sales.Customer
WHERE TerritoryID = 4;
-- NODO 2: Pacífico y Centro-Noreste
SELECT * FROM Sales.Customer
WHERE TerritoryID IN (9, 3, 2);

-- NODO 3: Europa Occidental
SELECT * FROM Sales.Customer
WHERE TerritoryID IN (10, 7);
```

```
-- NODO 4: Noroeste y Sureste
SELECT * FROM Sales.Customer
WHERE TerritoryID IN (1, 5);

-- NODO 5: Alemania y Canadá

SELECT * FROM Sales.Customer
WHERE TerritoryID IN (8, 6);
```

La Regla de Completitud y el Fragmento ***n+1***:

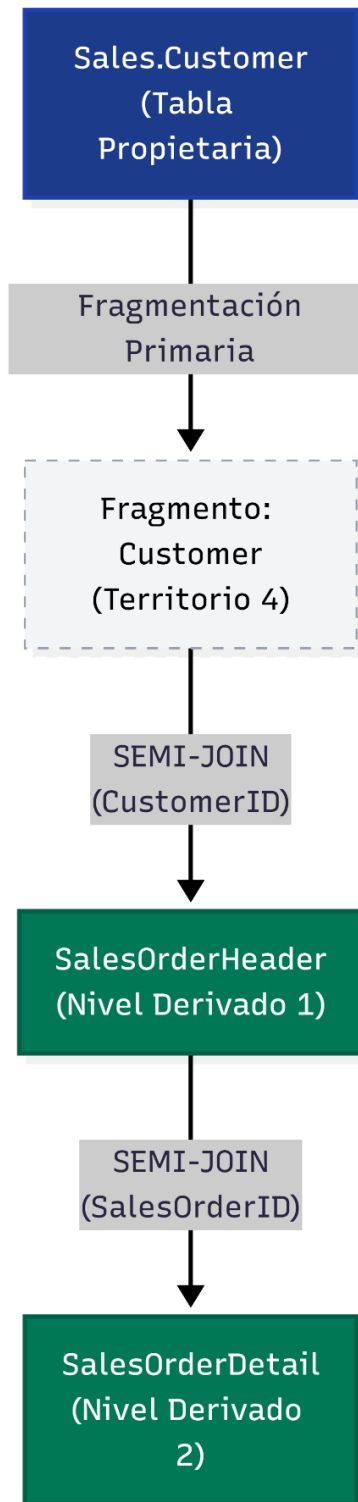
La teoría de Özsu y la regla de completitud dictan que ningún dato puede perderse en la fragmentación. ¿Si mañana la empresa abre un territorio nuevo (ej. TerritoryID = 11) o se inserta un cliente sin territorio asignado (NULL)? Nuestros 5 nodos principales no sabrían qué hacer con ellos.

Para solucionar esto de raíz, el diseño incluye un sexto fragmento, el Fragmento ***n+1***. Este fragmento funciona como una “red de seguridad” que atrapa cualquier territorio futuro o valor nulo

```
SELECT *
FROM Sales.Customer
WHERE TerritoryID NOT IN (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
OR TerritoryID IS NULL
```

Ahora, no solo consiste en clientes. Tenemos relaciones jerárquicas como las órdenes de venta (***SalesOrderHeader***) y detalles (***SalesOrderDetail***), las cuales reciben como llave foránea la llave del cliente (***CustomerID***).

Para mantener la eficiencia y evitar que los datos se separen de sus dueños, diseñamos el **grafo relacional**. Esto significa que aplicamos una **Fragmentación Derivada**.



La lógica es: *Si el Cliente 1 se guarda en el Nodo Norte, todas sus órdenes de compra y los detalles de esas compras heredan esa regla y se van al Nodo Norte.*

Para lograr esto sin traer columnas innecesarias o duplicar datos, utilizamos álgebra relacional **SEMI-JOIN** este SEMI-JOIN se traduce a la cláusula EXISTS. Esta técnica conecta la tabla hija con la tabla padre, propagando la fragmentación hacia abajo de manera eficiente

Validamos que nuestro diseño final cumpla con las 3 reglas de la fragmentación:

1. **Regla de Completitud:** Se cumple estrictamente gracias a nuestro fragmento adicional (n+1), garantizando que todos los registros (actuales, futuros y atípicos) tengan un nodo asignado.
2. **Regla de Reconstrucción:** Se cumple porque si aplicamos el operador de **UNION** sobre todos nuestros fragmentos (los 5 nodos + el fragmento n+1), podemos reconstruir la tabla global original de **Sales.Customer** sin pérdida de datos.
3. **Regla de Disyunción:** Se cumple porque un cliente solo puede tener un **TerritoryID**, por lo no es posible que un mismo registro resida en dos nodos a la vez.